Document Summary

This document covers the fundamentals of manipulator kinematics. It includes representations of position and orientation, rotation matrices, Euler angles, quaternions, homogeneous transformations, forward and inverse kinematics using the Denavit-Hartenberg Convention, differential kinematics with the Jacobian Matrix, kinematic singularities, and comparisons between serial and parallel manipulators. The focus is on the geometric and analytical mappings from joint configurations to poses and velocities, supported by detailed derivations and practical examples.

Kinematics of Robot Manipulators

Introduction to Robotics and Kinematics

Robotics

Robotics

As defined by J. M. Bradley from the MIT AI Lab in 1986, "The intelligent connection of perception to action," robotics involves the intelligent connection of perception to action.

This means that robotic systems integrate sensing (perception) with movement (action) in a smart way. For instance, examples such as META's CVPR 2023 demonstration and the MIT Media Lab's ICRA 2021 project show how visual perception can directly drive physical motions, effectively linking sensory input to coordinated movements.

Kinematics Overview

Kinematics

This branch of study examines the geometric and timing aspects of robot motion, while deliberately ignoring the forces involved. It primarily focuses on aspects like position and velocity.

To understand this better, consider that robots typically form a kinematic chain consisting of interconnected rigid bodies joined by joints, extending from a base to an end-effector. This

structure allows for coordinated movement across the system.

- ***Goal**: The main objective is to relate the motions at the joints to the overall motion of the robot. This involves mapping from a world reference frame to the robot's frame, such as determining the pose of the end-effector or the orientation of a mobile vehicle.*

Definitions

Parameterization q

Parameterization q

This provides an unambiguous and minimal way to characterize the configuration of the robot. Here, n represents the degrees of freedom (DoF), which equals the number of joints. The vector $\mathbf{q} = [q_1, q_2, \ldots, q_n]^T$ stores all the joint variables.

Parameterization x

Parameterization x

This describes the pose of the task, including position and orientation. The dimension $m \leq n$, where m is the dimension of the task space; this can differ in systems with redundancy.

Position and Orientation Representation

Position and Orientation Representation

These are essential for describing the poses of rigid bodies, allowing precise modeling of relationships between different frames.

Right-Hand Rule

Right-Hand Rule

This is a standard convention for defining rotations in 3D space: point your thumb in the direction of the rotation axis, and your fingers curl in the direction of positive rotation. It ensures consistent and predictable orientations in three-dimensional systems.

Position and Orientation of a Rigid Body

Characterization

Characterization

We start with a fixed world frame F_0 having origin O_0, and a body frame F_1 with origin O_1. The position and orientation are quantified relative to the world frame to fully describe the body's state.

Position

- The position vector from O_0 to O_1, expressed in F_0, is ${}^0\mathbf{p} = \begin{bmatrix} p_x & p_y & p_z \end{bmatrix}^T$.

Orientation

- **Representation**: The orientation is captured by the unit vectors along the axes of F_1, expressed in F_0. For example, the y-axis unit vector is ${}^0\mathbf{y}_1 = \begin{bmatrix} y_{1x} & y_{1y} & y_{1z} \end{bmatrix}^T$. The same applies to ${}^0\mathbf{x}_1$ and ${}^0\mathbf{z}_1$.
- These vectors are grouped into a *rotation matrix* ${}^0\mathbf{R}_1 = \begin{bmatrix} {}^0\mathbf{x}_1 & {}^0\mathbf{y}_1 & {}^0\mathbf{z}_1 \end{bmatrix}$.

Rotation Matrices

Properties

Properties

Rotation matrices are orthonormal, meaning their columns (and rows) are orthogonal to each other and have unit norm. They satisfy $\det(\mathbf{R}) = 1$ and belong to the Special Orthogonal Group SO(3). Importantly, the inverse is the transpose: $\mathbf{R}^{-1} = \mathbf{R}^T$.

Elementary Rotation Matrices

- For a rotation about the Z-axis by an angle θ:

Rotation about Z-axis

$$\mathbf{R}_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Similar matrices exist for rotations about the X-axis ($\mathbf{R}_x(\theta)$) and Y-axis ($\mathbf{R}_y(\theta)$).

Example: 90° Rotation about Z-axis

Consider $\theta = \frac{\pi}{2}$, where $\cos\theta = 0$ and $\sin\theta = 1$:

$$\mathbf{R}_z\left(\frac{\pi}{2}\right) = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Applying this to the vector $[1, 0, 0]^T$ yields $[0, 1, 0]^T$, representing a counterclockwise rotation in the XY plane. For the vector $[2, 0, 0]^T$, it results in $[0, 2, 0]^T$, demonstrating that the matrix preserves the vector's length.

Change of Coordinates of a Point

- For a point P with coordinates ${}^1\mathbf{p}$ in frame F_1, the coordinates in F_0 are given by ${}^0\mathbf{p} = {}^0\mathbf{R}_1^1\mathbf{p} + {}^0\mathbf{p}$, where ${}^0\mathbf{p}$ is the position of O_1. This formula combines both rotation and translation effects.
- The matrix ${}^0\mathbf{R}_1$ specifically encodes the rotation from F_1 to F_0.
- When dealing with frames that share the same origin, such as F_0, F_1, and F_2, the rotation composes as ${}^0\mathbf{R}_2 = {}^0\mathbf{R}_1^1\mathbf{R}_2$. Rotations thus compose multiplicatively, building up complex orientations from simpler ones.

Rotation of a Vector

- Consider a vector $\mathbf{v} = \begin{bmatrix} v_x & v_y & v_z \end{bmatrix}^T$; after rotation, it becomes $\mathbf{v}' = \mathbf{R}\mathbf{v}$. This changes the direction of the vector while preserving its magnitude.

Interpretations of a Rotation Matrix

- The matrix ${}^0\mathbf{R}_1(\theta)$ can be interpreted as the transformation that aligns frame F_0 with F_1.
- It also facilitates changes of coordinates without translation: ${}^0\mathbf{p} = {}^0\mathbf{R}_1^1\mathbf{p}$.
- Additionally, it describes the orientation of the body frame with respect to F_0.
- Finally, it can rotate vectors expressed in the same frame.

Representations of Orientation

- A rotation matrix uses 9 parameters but is subject to 6 constraints: 3 for unit norms (e.g., $x_{1x}^2 + x_{1y}^2 + x_{1z}^2 = 1$) and 3 for orthogonality (e.g., ${}^0\mathbf{x}_1 \cdot {}^0\mathbf{y}_1 = 0$). This makes it redundant; a minimal representation for $3D$ orientations requires only 3 parameters.
- Common methods include: the rotation matrix (9 parameters, intuitive for visualization); sequences of 3 elementary rotations (3 parameters); axis-angle representation (4 parameters: a unit axis vector plus an angle); and Quaternions (4 parameters, useful for interpolation and free of singularities).
- *Euler Angles*: These are defined as a sequence of elementary rotations relative to the current (body-fixed) frame.
- *Fixed Angles*: These involve rotations relative to a fixed world frame.

Euler Angles

> ***Euler Angles***
>
> *Any arbitrary orientation can be expressed as a sequence of elementary rotations, each performed with respect to the current body-fixed frame. This sequential approach builds the final orientation step by step.*

> ***Example: ZYZ Euler Angles***
>
> *This convention uses a rotation about Z, followed by a new Y axis, and then a new Z axis. The corresponding matrices are multiplied from left to right, reflecting the body-fixed sequence.*

> ***Example: Roll-Pitch-Yaw (ZYX Euler Angles)***
>
> *Common in aeronautics and mobile robotics, this involves a Z rotation (yaw), followed by Y (pitch), and then X (roll), all body-fixed.*

Fixed Angles

> ***Fixed Angles***
>
> *These represent orientations as a sequence of three elementary rotations, each with respect to the fixed world frame. This differs from Euler Angles in the reference frame used for each*

rotation.

Example: Fixed Angles XYZ

This applies a fixed Z rotation, then fixed Y, and fixed X. The matrices are multiplied from right to left. Notably, ZYX Euler angles and XYZ fixed angles result in the same mathematical expression.

Orientation Representation Conversions

- Standard formulas exist to convert between different representations, ensuring flexibility in computations.

Rotation Matrix to Fixed Angles XYZ

The angles are extracted as:

$$\phi_x = \atantwo(R_{32}, R_{33}), \quad \phi_y = -\arcsin(R_{31}), \quad \phi_z = \atantwo(R_{21}, R_{11})$$

Here, $\atantwo$ is used to obtain the correct quadrant for the angles, avoiding ambiguities.

Warning: Gimbal Lock Singularity

When the pitch angle $\theta = \pm 90°$, the roll and yaw axes align, leading to a loss of one degree of freedom. Rotations effectively collapse to a single axis, which can cause numerical instability in Euler or fixed angle representations near these orientations.

Example: Gimbal Lock Example

A visualization of this alignment degeneracy can be seen in the video at https://www.youtube.com/watch?v=zjMuIxRvygQ, which illustrates how the loss of freedom manifests in practice.

Unit Quaternion

Unit Quaternion

This representation avoids gimbal lock issues and uses 4 parameters (though non-minimal due to the unit norm constraint). It is particularly useful for smooth interpolation between orientations.

- A quaternion is expressed as $\mathbf{q} = w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k}$, with vector part $\mathbf{v} = [x, y, z]^T$, satisfying the unit norm $w^2 + x^2 + y^2 + z^2 = 1$.
- **Operations**:
  - Addition is performed component-wise on (w, x, y, z).
  - Multiplication follows $\mathbf{q}_1\mathbf{q}_2 = (w_1w_2 - \mathbf{v}_1 \cdot \mathbf{v}_2) + (w_1\mathbf{v}_2 + w_2\mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2)$.
  - The additive identity is $0 + 0\mathbf{i} + 0\mathbf{j} + 0\mathbf{k}$.
  - The multiplicative identity is $1 + 0\mathbf{i} + 0\mathbf{j} + 0\mathbf{k}$.
- The conjugate is $\mathbf{q}^* = w - x\mathbf{i} - y\mathbf{j} - z\mathbf{k}$, and for unit quaternions, $\|\mathbf{q}^*\| = 1$.
- **Conversions**:

> ***Quaternion to Rotation Matrix***
>
> *The corresponding rotation matrix is:*
>
> $$\mathbf{R} = \begin{bmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{bmatrix}$$

- **Inverse**: For a general quaternion, $\mathbf{q}^{-1} = \mathbf{q}^*/\|\mathbf{q}\|^2$; for unit quaternions, it simplifies to $\mathbf{q}^{-1} = \mathbf{q}^*$.
- **Rotation Matrix to Quaternion**: This involves extracting the trace and elements, followed by square roots. For example, $w = \frac{1}{2}\sqrt{1 + R_{11} + R_{22} + R_{33}}$. Adjustments are made for numerical stability by selecting the largest component.

> ***Example: 90° Rotation about Z-axis with Quaternion***
>
> *The quaternion is $\mathbf{q} = \frac{\sqrt{2}}{2} + 0\mathbf{i} + 0\mathbf{j} + \frac{\sqrt{2}}{2}\mathbf{k}$. Converting this yields $\mathbf{R}_z\left(\frac{\pi}{2}\right)$, where $w, z \approx 0.707$ matches the matrix elements like $R_{11} = 0$, $R_{12} = -1$, and so on.*

Homogeneous Representation

- The basic transformation for position is ${}^0\mathbf{p} = {}^0\mathbf{R}_1^1\mathbf{p} + {}^0\mathbf{p}$, with the inverse ${}^1\mathbf{p} = \mathbf{R}_1^T({}^0\mathbf{p} - {}^0\mathbf{p})$.
- In homogeneous coordinates, points are augmented as $\tilde{\mathbf{p}} = \begin{bmatrix}\mathbf{p}\\1\end{bmatrix}$, and the full transformation is the matrix ${}^0\mathbf{T}_1 = \begin{bmatrix}{}^0\mathbf{R}_1 & {}^0\mathbf{p}\\ \mathbf{0}^T & 1\end{bmatrix}$.
- The inverse transformation is ^1\mathbf{T}_0 = \begin{bmatrix} \mathbf{R}_1^T & -\mathbf{R}_1^T {}^0\mathbf{p} \\ \mathbf{0}^T & 1 \end{bmatrix}.
- This 4×4 matrix form greatly simplifies the chaining of multiple transformations through straightforward matrix multiplication, making it ideal for kinematic chains.

Time Dependent Rotations

- Assume frame F_0 is stationary while F_1 rotates over time. For a point fixed in F_1, its coordinates ${}^1\mathbf{p}$ remain constant, so ${}^0\mathbf{p}(t) = {}^0\mathbf{R}_1(t){}^1\mathbf{p}$.
- To analyze motion, differentiate with respect to time. From the property ${}^0\mathbf{R}_1{}^0\mathbf{R}_1^T = \mathbf{I}$, differentiating yields ${}^0\dot{\mathbf{R}}_1{}^0\mathbf{R}_1^T$, which is a skew-symmetric matrix.
- This leads to the angular velocity matrix: ${}^0\boldsymbol{\omega}_1 = {}^0\dot{\mathbf{R}}_1{}^0\mathbf{R}_1^T$,

> ***Angular Velocity Matrix***
>
> $$ {}^0\boldsymbol{\omega}_1 = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix} $$
>
> *where $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^T$ is the angular velocity vector.*

- The time derivative of the rotation matrix is ${}^0\dot{\mathbf{R}}_1 = {}^0\boldsymbol{\omega}_1{}^0\mathbf{R}_1$.
- The matrix acts on vectors as ${}^0\boldsymbol{\omega}_1\mathbf{v} = \boldsymbol{\omega} \times \mathbf{v}$, mimicking the cross product.
- For linear velocity, ${}^0\mathbf{v} = \boldsymbol{\omega} \times {}^0\mathbf{p}$, describing how points move due to rotation.

Kinematics of Serial Manipulators

- Serial manipulators consist of links arranged in a single chain, such as a robotic arm. The first three degrees of freedom are often realized through prismatic or revolute joints, allowing translation or rotation.

Forward Kinematics

Forward Kinematics

This computes the pose of the end-effector from the joint variables: $\mathbf{x} = f(\mathbf{q})$, mapping from joint space to task space.

Example: 2-Link Planar Arm by Inspection

For a 2-link planar arm with link lengths a_1 and a_2, and joint angles q_1 and q_2, the end-effector position is:

$$p_x = a_1 \cos q_1 + a_2 \cos(q_1 + q_2), \quad p_y = a_1 \sin q_1 + a_2 \sin(q_1 + q_2)$$

Using $a_1 = a_2 = 1$, $q_1 = 30^\circ \approx 0.524$ rad, and $q_2 = 45^\circ \approx 0.785$ rad, the position calculates to $p_x \approx 1.62$ and $p_y \approx 1.25$. This direct geometric derivation shows how joint angles determine the reachable workspace.

Towards a Systematic Approach

- To generalize, assign a coordinate frame to each link, where the transformation ${}^{i-1}\mathbf{T}_i$ depends on the variable q_i.
- The overall transformation from base to end-effector is ${}^0\mathbf{T}_n = {}^0\ \mathbf{T}_1 {}^1\mathbf{T}_2 \cdots {}^{n-1}\ \mathbf{T}_n$.
- Standardizing the frame assignments ensures consistency and ease of computation across different manipulators.

Denavit-Hartenberg Convention

Denavit-Hartenberg Convention

This convention standardizes the placement of coordinate frames for each link using four parameters per link: a_i (link length, fixed), α_i (link twist, fixed), d_i (link offset, variable for prismatic joints), and θ_i (joint angle, variable for revolute joints).

DH Convention Rules

1. *The z_i axis is aligned with the axis of joint $i+1$.*
2. *The origin O_i is placed at the intersection of the z_{i+1} axis with the common normal between z_i and z_{i+1}; if the axes are parallel, place it at any point along z_i (denoted O_i').*
3. *The x_i axis runs along the common normal, directed from z_i toward z_{i+1}.*

4. The y_i axis completes the right-handed coordinate system.

- **Transformation F_{i-1} to F_i:** This is achieved through sequence of roto-translations:
  1. Translate along z_{i-1} by d_i to reach an intermediate point O_i'.
  2. Rotate about z_{i-1} by θ_i to align x_{i-1} with the common normal, forming x_i.
  3. Translate along x_i by a_i to reach O_i.
  4. Rotate about x_i by α_i to align with z_i.

Example: DH Parameters Example

A demonstration of frame placement using the DH convention can be viewed at https://www.youtube.com/watch?v=rA9tm0gTln8, which walks through assigning frames step by step for a sample manipulator.

DH Homogeneous Transformation

- *First, translate along z_{i-1} by d_i and rotate by θ_i:*

$$\mathbf{A}_z = \begin{bmatrix} \cos\theta_i & -\sin\theta_i & 0 & 0 \\ \sin\theta_i & \cos\theta_i & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- *Then, translate along x_i by a_i and rotate by α_i:*

$$\mathbf{A}_x = \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & \cos\alpha_i & -\sin\alpha_i & 0 \\ 0 & \sin\alpha_i & \cos\alpha_i & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- *The combined transformation is ${}^{i-1}\mathbf{T}_i = \mathbf{A}_z\mathbf{A}_x$:*

$${}^{i-1}\mathbf{T}_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

  - *For revolute joints, the variable is $q_i = \theta_i$; for prismatic joints, it is $q_i = d_i$.*

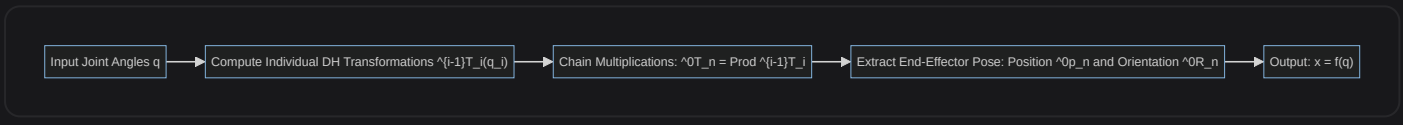
- The full forward kinematics is ^0\mathbf{T}_n = \prod_{i=1}^n ^{i-1}\mathbf{T}_i(q_i); from this, extract the rotation ${}^0\mathbf{R}_n$ and position ${}^0\mathbf{p}_n$.

> ***Example: 2-Link Planar Arm with DH***
>
> *Setting $\alpha_i = d_i = 0$, $a_1 = a_2 = 1$, $q_1 = 30^\circ$, $q_2 = 45^\circ$ yields $p_x \approx 1.62$ and $p_y \approx 1.25$, matching the inspection method.*

Joint	α_i	d_i	a_i	θ_i
1	0	0	a_1	q_1
2	0	0	a_2	q_2

Forward Kinematics and ROS

- In practice, forward kinematics can be implemented using ROS (Robot Operating System) with URDF (Unified Robot Description Format) files. The TF (Transform) package then computes and broadcasts the poses automatically.



Inverse Kinematics

> ***Inverse Kinematics***
>
> *This solves for the joint variables given a desired end-effector pose: $\mathbf{q} = f^{-1}(\mathbf{x})$. Solutions may not exist, or there could be multiple valid ones.*

Multiplicity of Solutions

- In forward kinematics, a unique $\mathbf{x}$ results from any $\mathbf{q}$.
- In inverse kinematics, possibilities include zero solutions (if the pose is unreachable), multiple finite solutions (due to geometric configurations), or infinite solutions (in redundant systems).

Possible Cases

- For non-redundant manipulators ($m = n$):
  1. No solutions exist outside the workspace.
  2. A finite number of solutions, such as the "elbow up" or "elbow down" configurations in planar arms.
  3. At singularities, solutions may be finite or infinite, particularly at workspace boundaries.

- For redundant manipulators ($m < n$):
  1. No solutions for unreachable poses.
  2. Infinite solutions, parameterized by ∞^{n-m} degrees of freedom.
  3. Singularities further constrain the solution set.

> ***Example: Non-redundant Example***
>
> *A 6-DoF industrial arm ($m = n = 6$) can have up to 8 possible solutions for a given pose, though some may be invalid due to joint limits or collision risks.*

Solving the Inverse Kinematics

- **Analytical Solution (Closed Form)**: This involves explicit algebraic or geometric formulas derived for the specific manipulator geometry. It is fast and allows enumeration of all solutions, making it suitable for simpler designs.
- **Numerical Solution (Iterative)**: This uses optimization techniques to minimize the error $\|\mathbf{x} - f(\mathbf{q})\|$, such as the Newton-Raphson method or gradient descent. It is more general for complex redundant systems but can be slower and risks converging to local minima.

> ***Example: Analytical Solution Example (2R Arm)***
>
> *For a desired position (p_x, p_y) in a 2R planar arm:*
>
> $$\theta_2 = \pm \acos\left(\frac{p_x^2 + p_y^2 - a_1^2 - a_2^2}{2a_1 a_2}\right)$$
>
> *This gives the two elbow configurations (up or down). Then, solve for:*
>
> $$\theta_1 = \atantwo(p_y, p_x) - \atantwo(a_2 \sin\theta_2, a_1 + a_2 \cos\theta_2)$$
>
> *For $p_x = 1.5$, $p_y = 1$, and $a_1 = a_2 = 1$, one solution is $\theta_1 \approx 45^\circ$, $\theta_2 \approx 30^\circ$; the other flips the sign of θ_2.*

> ***Process: Numerical Solution: Newton Method***
>
> 1. *Define the error function $\mathbf{g}(\mathbf{q}) = \mathbf{x} - f(\mathbf{q}) = \mathbf{0}$.*
> 2. *Compute the Jacobian $\mathbf{J}_a = \frac{\partial f}{\partial \mathbf{q}}$.*
> 3. *Update iteratively: $\mathbf{q}_{k+1} = \mathbf{q}_k - \mathbf{J}_a^{-1}\mathbf{g}(\mathbf{q}_k)$.*
> 4. *For redundant cases ($m < n$), use the pseudoinverse $\mathbf{J}_a^+$ instead of the full inverse.*

Process: Numerical Solution: Gradient Method

1. Define the cost function $h(\mathbf{q}) = \frac{1}{2}\|\mathbf{x} - f(\mathbf{q})\|^2$.

2. Compute the gradient $\nabla h = -\mathbf{J}_a^T(\mathbf{x} - f(\mathbf{q}))$.

3. Update: $\mathbf{q}_{k+1} = \mathbf{q}_k + \alpha \mathbf{J}_a^T(\mathbf{x} - f(\mathbf{q}_k))$, *where* $\alpha > 0$ *is the step size.*

Newton vs. Gradient Method

Gradient Method

- *It is simple to implement since it only requires the transpose* $\mathbf{J}^T$, *avoiding matrix inversion.*
- *It prevents divergence but converges linearly, which can be slow near the solution.*

Newton Method

- *Computationally intensive due to matrix inversion.*
- *Offers quadratic convergence near the solution but may not be globally reliable.*

- **Strategy**: Often, use the gradient method for initial coarse approximation, followed by the Newton method for precise fine-tuning.

Differential Kinematics

- Differential kinematics relates the end-effector twist (linear velocity $\mathbf{v}$ and angular velocity $\boldsymbol{\omega}$) to the joint velocities $\dot{\mathbf{q}}$ via $\begin{bmatrix}\mathbf{v}\\ \boldsymbol{\omega}\end{bmatrix} = \mathbf{J}\dot{\mathbf{q}}$, where $\mathbf{J}$ is the *Geometric Jacobian*. This is crucial for velocity-based control and handling constraints.

Geometrical Jacobian

- The Jacobian is a $6 \times n$ matrix, with columns representing the twist contributions from each joint.
- For the i-th joint, the linear velocity component is $\mathbf{v}_i = \boldsymbol{\omega}_i \times \mathbf{p}$, where $\mathbf{p} = {}^0\mathbf{p}_n - {}^0\mathbf{p}_{i-1}$.
- The full Jacobian is $\mathbf{J} = [\mathbf{j}_1, \ldots, \mathbf{j}_n]$, where $\mathbf{j}_i = \begin{bmatrix}\mathbf{v}_i/\dot{q}_i\\ \boldsymbol{\omega}_i/\dot{q}_i\end{bmatrix}$.
- The angular velocity $\boldsymbol{\omega}_i$ depends on the joint type.

Prismatic Joints

> ***Prismatic Joints***
>
> - *The angular velocity is* $\boldsymbol{\omega}_i = \mathbf{0}$.
> - *The linear velocity is* $\mathbf{v}_i = {}^0\mathbf{z}_{i-1}$ *(scaled by* $\dot{q}_i$*, the translation rate).*

Rotational Joints

> ***Rotational Joints***
>
> - *The angular velocity is* $\boldsymbol{\omega}_i = {}^0\mathbf{z}_{i-1}$ *(scaled by* $\dot{q}_i$*, the rotation rate).*
> - *The linear velocity is* $\mathbf{v}_i = {}^0\mathbf{z}_{i-1} \times ({}^0\mathbf{p}_n - {}^0\mathbf{p}_{i-1})$ *(also scaled by* $\dot{q}_i$*). These are derived from the DH parameters.*

- In column form: for prismatic joints, $\mathbf{j}_i = \begin{bmatrix} {}^0\mathbf{z}_{i-1} \\ \mathbf{0} \end{bmatrix}$; for rotational joints, $\mathbf{j}_i = \begin{bmatrix} {}^0\mathbf{z}_{i-1} \times ({}^0\mathbf{p}_n - {}^0\mathbf{p}_{i-1}) \\ {}^0\mathbf{z}_{i-1} \end{bmatrix}$.

> ***Example: Planar 2R Arm Jacobian***
>
> *For position only, the* 2×2 *Jacobian is:*
>
> $$\mathbf{J} = \begin{bmatrix} -a_1 \sin q_1 - a_2 \sin(q_1 + q_2) & -a_2 \sin(q_1 + q_2) \\ a_1 \cos q_1 + a_2 \cos(q_1 + q_2) & a_2 \cos(q_1 + q_2) \end{bmatrix}$$
>
> *(Here,* $\mathbf{p}_1 = [a_1 \cos q_1, a_1 \sin q_1]^T$ *and* $\mathbf{p}_2 = [p_x, p_y]^T$*).*
>
> *With* $q_1 = 30^\circ$*,* $q_2 = 45^\circ$*, and* $a_1 = a_2 = 1$*,* $\mathbf{J} \approx \begin{bmatrix} -1.366 & -0.707 \\ 1.366 & 0.707 \end{bmatrix}$*. For* $\dot{\mathbf{q}} = [1, 1]^T$ *rad/s, the end-effector velocity is* $\mathbf{J}\dot{\mathbf{q}} \approx [-2.07, 2.07]^T$ *m/s.*

Analytical vs. Geometric Jacobian

- The *Geometric Jacobian* $\mathbf{J}$ maps $\dot{\mathbf{q}}$ to the twist $(\mathbf{v}, \boldsymbol{\omega})$, providing an intuitive physical interpretation.
- The *Analytical Jacobian* $\mathbf{J}_a = \frac{\partial f}{\partial \mathbf{q}}$ maps to task-space derivatives, including $\dot{\boldsymbol{\phi}}$ (angular velocity in Euler angles) rather than $\boldsymbol{\omega}$.
- The relationship is $\boldsymbol{\omega} = \mathbf{T}\dot{\boldsymbol{\phi}}$, where for ZYZ Euler angles:

Transformation Matrix T for ZYZ Euler Angles

$$\mathbf{T} = \begin{bmatrix} -\sin\phi_3 & \cos\phi_3 \sin\phi_2 & \cos\phi_3 \cos\phi_2 \\ \cos\phi_3 & -\sin\phi_3 \sin\phi_2 & \sin\phi_3 \cos\phi_2 \\ 0 & \cos\phi_2 & -\sin\phi_2 \end{bmatrix}$$

The analytical Jacobian $\mathbf{J}_a$ is used when the task involves derivatives in Euler angles, relating $\dot{\mathbf{x}}$ (including $\dot{\boldsymbol{\phi}}$) to $\dot{\mathbf{q}}$.

Kinematic Singularities

- The velocity mapping $\dot{\mathbf{x}} = \mathbf{J}\dot{\mathbf{q}}$ becomes singular when the Jacobian $\mathbf{J}$ is rank-deficient (e.g., $\det(\mathbf{J}) = 0$ or the smallest singular value $s_{\min} \approx 0$): these are *kinematic singularities*.

Singularities

- *They result in a loss of instantaneous mobility, making certain velocity directions impossible for the end-effector.*
- *Infinite solutions may exist for inverse velocity mappings.*
- *Small end-effector velocities can require excessively high joint velocities.*
- *Singularities often occur at the workspace boundary (e.g., fully extended arm) or internally (e.g., aligned joints).*

Example: Singularity Examples

In a 6-DoF arm, a shoulder singularity happens when the arm is fully extended or folded, blurring the distinction between pitch and roll motions. A visualization is available at https://youtu.be/lD2HQcxeNoA?si=6ayDcdU8CINOmsjt.

Inversion of Differential Kinematics

- For square and invertible Jacobians, $\dot{\mathbf{q}} = \mathbf{J}^{-1}\dot{\mathbf{x}}$.
- For redundant or non-square cases, use $\dot{\mathbf{q}} = \mathbf{J}^{+}\dot{\mathbf{x}} + (\mathbf{I} - \mathbf{J}^{+}\mathbf{J})\mathbf{z}$, where $\mathbf{z}$ is an arbitrary vector in the null space (useful for secondary tasks like obstacle avoidance).
- The pseudoinverse is $\mathbf{J}^{+} = \mathbf{V}\boldsymbol{\Sigma}^{-1}\mathbf{U}^{T}$ from the SVD $\mathbf{J} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{T}$.
- Near singularities ($s_{\min} \approx 0$), the term $1/s_{\min}$ amplifies $\dot{\mathbf{q}}$, leading to instability; damping or regularization techniques are applied to mitigate this.

Parallel Manipulators

- Parallel manipulators feature multiple kinematic chains connecting the base to the end-effector, such as hexapod walkers or Stewart platforms. They offer high stiffness and precision, ideal for applications like flight simulators or precision assembly.

How a Parallel Robot Moves?

> ***Example: Hexapod Platforms***
>
> *A hexapod uses 6 actuated legs to control the end-effector pose through extensions and contractions of the legs. The attachment points are ${}^{b}\mathbf{a}_i$ on the body frame, and the overall pose is ${}^{w}\mathbf{T}_b$. The leg vectors extend from base attachment points to body points, determining joint positions.*

Inverse Kinematics

> ***Inverse Kinematics for Parallel Robots***
>
> *Unlike serial chains, inverse kinematics is often straightforward, as each leg can be solved independently.*

- This relies on *loop-closure* equations: the path from base to end-effector via each leg must close the loop.
- For leg i, ${}^{w}\mathbf{p}_i = {}^{w}\mathbf{T}_b \begin{bmatrix} {}^{b}\mathbf{a}_i \\ 1 \end{bmatrix} + \rho_i {}^{w}\mathbf{u}_i$, where ρ_i is the joint variable (leg length) and ${}^{w}\mathbf{u}_i$ is the direction vector.
- With fixed ${}^{b}\mathbf{a}_i$ and a desired ${}^{w}\mathbf{T}_b$, solve geometrically for each ρ_i to obtain the joint vector $\rho = [\rho_1, \ldots, \rho_n]$.

Forward Kinematics

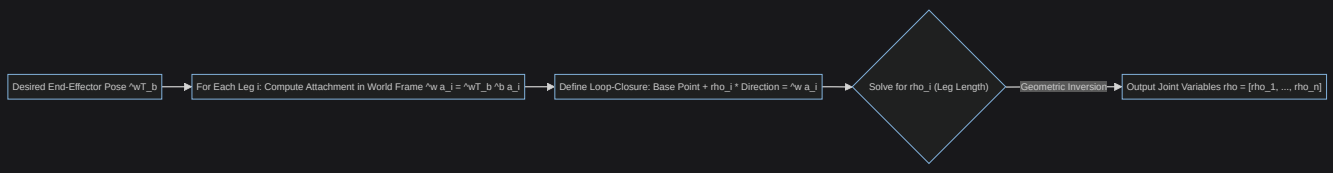
- Forward kinematics is more complex due to the coupled nonlinear equations from all loop-closures, often yielding multiple discrete solutions for the pose.

> ***Example: Stewart (Gough) Platform***

This 6-DoF parallel manipulator has 6 extensible legs; given leg lengths, there can be up to 40 possible poses, from which the feasible one is selected based on constraints.

Example: 2P Planar Parallel Robot

With two equal-length prismatic legs fixed at the base, a desired end-effector position can yield multiple configurations: symmetric (uncrossed) or crossed legs.



References

- Robotics
- Denavit-Hartenberg Convention
- Jacobian Matrix
- Euler Angles
- Quaternions
- Linear Algebra